

SEMANA 3 ACADEMIA MONTERREY

- Seguridad y autenticación

Los temas de seguridad y autenticación se tomaron el día 2 y 3 de la semana 3 de la academia. Se trata de la seguridad de los usuarios y desarrolladores con respecto a su aplicación y la forma en que se administran las bases de datos disponibles. Por ejemplo, ¿cómo podemos hacer para que nuestra aplicación sepa quién tiene permisos de editar, acceder, o de manera general conocer los datos de nuestros usuarios? Es aquí donde entran los siguientes conceptos:

1. Distinguir la diferencia entre “¿Quién eres?” y “¿Qué puedes hacer?”. Estos dos procesos identifican a la entidad que realiza la llamada y verifican si el usuario identificado tiene permiso para ejecutar una acción, respectivamente.
2. Qué formas hay para implementar protocolos de seguridad e identificación en nuestras aplicaciones Spring. Tomamos en cuenta tres diferentes técnicas y/o herramientas relevantes: HTTP Basic, JWT, y OAuth.

● HTTP Basic

Mediante este método, las peticiones atraviesan filtros especializados antes de llegar al controlador. El filtro inicial verifica las credenciales en Base64 y compara los hashes BCrypt, mientras que el filtro posterior valida los roles del usuario frente a las reglas de acceso. Cada petición debe incluir de forma independiente las credenciales de identidad en la cabecera HTTP. El servidor no almacena ningún estado de sesión entre peticiones. Las contraseñas no se almacenan en texto plano. BCrypt incorpora un código aleatorio de 22 caracteres directamente en la estructura del hash resultante, lo que provoca que contraseñas idénticas generen hashes únicos. Está diseñado intencionadamente para ser computacionalmente lento, con el fin de evitar ataques de fuerza bruta mediante GPU.

La mayor desventaja de usar HTTP Basic es que la codificación Base64 no está cifrada, es decir, es fácilmente reversible. Transmitir credenciales en cada solicitud sin el protocolo HTTPS expone dichas credenciales en texto plano a herramientas de captura de tráfico, proxies y registros de acceso.

En el programa presentado relacionado a HTTP Basic, el controlador es definido por `DispatcherServlet/EmployeeRestController`. La seguridad se incorpora mediante dos componentes `@Bean` principales en `SecurityConfig.java`: consultas SQL personalizadas a través de `JdbcUserDetailsManager` para recuperar usuarios (miembros) y permisos (roles), y una cadena de filtros HTTP para establecer los privilegios asociados a los roles.

- **JWT**

Las siglas JWT significan JSON Web Tokens. Mientras que en HTTP Basic las credenciales se envían en cada solicitud, JWT sustituye este método utilizando las credenciales una sola vez en un punto final de inicio de sesión para obtener un pase de acceso firmado. Las solicitudes posteriores a puntos finales protegidos solo incluyen el token en la cabecera, es decir, un pase. Validar un JWT requiere comprobar su firma digital en memoria mediante una clave pública; no implica consultar la base de datos ni ejecutar verificaciones BCrypt en las solicitudes posteriores

Un JWT se divide en tres segmentos codificados en Base64 y separados por puntos:

1. Encabezado (Header): Especifica el algoritmo de firma
2. Carga útil o payload (Claims): Contiene metadatos de identidad no sensibles, tales como el emisor (iss), el sujeto (sub), la marca de tiempo de emisión (iat), la marca de tiempo de expiración (exp) y ámbitos o roles personalizados (roles).
3. Firma (Signature): Un hash criptográfico generado a partir del encabezado y la carga útil codificados, utilizando la clave privada RSA.

En nuestro código, se implementan las siguientes declaraciones Bean para implementar los codificadores y decodificadores JWT:

```
// Signs outgoing tokens at login using the private key
```

```
@Bean
```

```
public JwtEncoder jwtEncoder() {
```

```
    JWK jwk = new RSAKey.Builder(publicKey).privateKey(privateKey).build();
```

```
    return new NimbusJwtEncoder(new ImmutableJWKSet<>(new JWKSet(jwk)));
```

```
}
```

```
// Validates incoming Bearer tokens using the public key
```

```
@Bean
```

```
public JwtDecoder jwtDecoder() {  
    return NimbusJwtDecoder.withPublicKey(publicKey).build();  
}
```

Así mismo, la identificaciones se realiza a partir del siguiente código, el cual se encarga de codificar las peticiones del token:

```
@PostMapping("/login")  
public TokenResponse login(Authentication authentication) {  
    Instant now = Instant.now();  
  
    // Filter internal Spring Security 7 authority flags (e.g., FACTOR_PASSWORD)  
    List<String> roles = authentication.getAuthorities().stream()  
        .map(GrantedAuthority::getAuthority)  
        .filter(auth -> auth.startsWith("ROLE_"))  
        .toList();  
  
    JwtClaimsSet claims = JwtClaimsSet.builder()  
        .issuer("security-jwt")  
        .issuedAt(now)  
        .expiresAt(now.plusSeconds(ttlSeconds))  
        .subject(authentication.getName())  
        .claim("roles", roles)  
        .build();  
  
    String tokenValue = jwtEncoder.encode(JwtEncoderParameters.from(claims)).getTokenValue();  
    return new TokenResponse(tokenValue, "Bearer", ttlSeconds, authentication.getName());  
}
```

- **OAuth2**

Con la utilización del método OAuth2, En lugar de almacenar contraseñas de usuario y emitir tokens localmente, la aplicación delega la gestión de identidades a un servicio dedicado o tercero, en nuestro caso, Keycloak. La API ya no pregunta "¿cuál es tu contraseña?", sino más bien "¿quién te avala?". Se validan las firmas de los JWT entrantes utilizando claves públicas descargadas desde el endpoint de JSON Web Key Set (JWKS) de Keycloak (`jwks\_uri`),

eliminando la necesidad de comunicarse con Keycloak en cada solicitud. De manera resumida, OAuth2 se centra en la autorización: Emite un `access\_token` que los clientes utilizan para acceder a recursos protegidos de la API.

También se incorpora el uso de OpenID Connect (OIDC). Este protocolo añade una capa de autenticación sobre OAuth2 emitiendo un `id\_token` que contiene detalles del perfil de usuario (nombre, correo electrónico) destinados al frontend, no a la API del backend.

La utilidad de implementar OAuth2 es que delegamos la responsabilidad de manejar contraseñas y permisos a un tercero en lugar de cuidar nosotros mismos la información correspondiente, lo cuál resulta útil y a la vez provee seguridad al conectar nuestras aplicaciones con diversos servicios.